

Tracking Performance Drift in Large Language Models Across Successive Version Updates

Abstract

Large language models (LLMs) deployed as commercial application programming interfaces (APIs) and consumer chat products are updated silently and frequently, yet the empirical consequences of these updates for downstream task performance remain poorly characterized. This paper synthesizes and extends the emerging literature on “performance drift” — unannounced, often non-monotonic changes in model behavior across successive versions of the “same” LLM service. Building on longitudinal audits such as Chen et al. (2023), ChatLog (Tu et al., 2024), and related contamination and evaluation-integrity studies, we organize the discussion around four themes: (1) the empirical evidence for drift across capability, safety, and stylistic dimensions; (2) the technical mechanisms — continued fine-tuning, reinforcement learning from human feedback (RLHF) updates, quantization, system-prompt changes, and infrastructure substitutions — that plausibly produce drift; (3) current monitoring and evaluation approaches, including holistic benchmarking frameworks, LLM-as-a-judge protocols, and concept-drift-inspired statistical detectors; and (4) the methodological and infrastructural challenges that impede rigorous, reproducible drift tracking, including provider opacity, benchmark contamination, and the absence of standardized longitudinal reporting. We identify concrete research gaps — the lack of causal attribution methods linking specific update mechanisms to observed behavioral shifts, the absence of shared longitudinal infrastructure analogous to software regression-testing suites, and insufficient attention to drift in non-English and multimodal settings — and propose directions for future work, including standardized drift-reporting schemas modeled on model cards, community-maintained “canary” benchmarks, and causal auditing frameworks. We argue that performance drift is not merely a curiosity of black-box APIs but a first-order reliability concern for any organization that builds production systems atop continuously evolving foundation models.

Keywords

large language models; performance drift; model versioning; longitudinal evaluation; benchmark contamination; RLHF; concept drift; LLM monitoring; reproducibility

1. Introduction

The rapid commercialization of large language models has created a software ecosystem in which the fundamental computational artifact underlying a product — the model itself — changes on a schedule that is opaque to, and largely uncontrolled by, the developers who build on top of it. Unlike traditional software libraries, which are versioned explicitly and whose changes are documented in release notes, commercial LLM endpoints such as those exposed by OpenAI, Anthropic, and Google are frequently updated behind a stable API name or model alias, with the timing and content of updates disclosed only partially, or not at all. Chen, Zaharia, and Zou (2023) were among the first to systematically document the consequences of this practice, showing that GPT-3.5 and GPT-4 accessed through ChatGPT exhibited large, sometimes contradictory, shifts in accuracy on tasks such as prime-number identification, code

generation, and multi-hop question answering between March 2023 and June 2023 snapshots of the “same” models.

This phenomenon, which we refer to broadly as performance drift, is distinct from the well-studied problem of data drift or concept drift in classical machine learning, where a static model's predictions degrade because the distribution of incoming data shifts away from the training distribution (Gama et al., 2014; Lu et al., 2019). In the LLM setting, the model itself is the moving target: providers continuously fine-tune, safety-tune, quantize, and re-serve their models, and downstream users have no reliable way to know whether a given API call today will be answered by the same underlying weights, decoding configuration, and system prompt as it was answered yesterday. The result is a form of drift that is simultaneously a machine learning phenomenon (changes in learned behavior) and a software engineering phenomenon (changes in a dependency that downstream systems silently inherit).

Tracking this drift matters for at least three reasons. First, applications built on LLMs — for example, retrieval-augmented question answering systems, coding assistants, or automated content moderation pipelines — can break or silently degrade when the underlying model's output format, refusal behavior, or reasoning strategy changes, even though no code in the application itself was modified (Chen et al., 2023). Second, drift complicates scientific reproducibility: published evaluations of a named model (e.g., “GPT-4”) may not be replicable weeks later because the model accessed under that name has since changed (Aiyappa et al., 2023). Third, drift interacts with benchmark integrity: because providers continue to train on data collected after a model's initial release, and because evaluation datasets themselves may leak into subsequent training corpora, apparent performance “improvements” across versions can reflect contamination rather than genuine capability gains (a survey of these dynamics is provided by the contamination-detection literature; see Section 2).

This paper reviews the current state of research on tracking LLM performance drift across successive version updates, synthesizes the technical mechanisms believed to drive it, catalogs current monitoring and evaluation approaches, and identifies open research gaps. The remainder of the paper is organized as follows. Section 2 reviews related work in longitudinal LLM evaluation, classical concept drift, and model documentation. Section 3 provides a technical discussion of the mechanisms and dimensions of drift. Section 4 surveys current monitoring approaches and tooling. Section 5 discusses research challenges and limitations of existing work. Section 6 proposes research gaps and future directions. Section 7 concludes.

2. Background, Related Work, and Literature Review

2.1 Foundational Longitudinal Audits of Commercial LLMs

The best-known empirical study of LLM drift is Chen, Zaharia, and Zou's (2023) evaluation of the March 2023 and June 2023 releases of GPT-3.5 and GPT-4, later published in the Harvard Data Science Review (Chen et al., 2024). Using identical prompts across seven task families — arithmetic reasoning (prime number identification), sensitive or potentially dangerous questions, opinion surveys, multi-hop knowledge questions, code generation, medical licensing examination questions, and visual reasoning — the authors found that GPT-4's accuracy on prime-number identification fell from roughly 84% in March

2023 to roughly 51% in June 2023, while GPT-4's willingness to directly answer sensitive questions dropped sharply over the same period, and both models' code outputs became substantially less directly executable due to added non-code formatting artifacts such as triple-quote wrapping. Critically, some capabilities improved over the same interval (e.g., GPT-4's multi-hop question answering), demonstrating that drift is not uniformly degenerative: models can simultaneously improve on some axes and regress on others, a pattern the authors attribute in part to changes in instruction-following fidelity.

A complementary longitudinal effort is ChatLog (Tu et al., 2024), which constructed an ever-updating dataset of ChatGPT responses to 21 NLP benchmarks collected monthly, alongside a daily-updated set of 1,000 fixed prompts for long-form generation. In contrast to Chen et al.'s finding of large, task-specific swings, Tu et al. report that most ChatGPT capabilities improved gradually over time with a “step-wise” evolution pattern, while identifying specific abilities that regressed. ChatLog additionally shows that certain linguistic and stylistic features of ChatGPT's output remain stable across versions even as task accuracy shifts, and the authors use these invariant features to build a more version-robust classifier for detecting machine-generated text — illustrating that drift tracking has downstream utility beyond simple performance monitoring.

Earlier and concurrent work by Chen, Cai, Zaharia, and Zou (2021) examined a related but distinct problem: silent behavioral changes in commercial machine-learning-as-a-service (MLaaS) APIs more broadly, proposing efficient statistical methods for detecting whether an API's outputs had shifted without requiring exhaustive re-evaluation. This line of work situates LLM drift within a broader tradition of auditing black-box, continuously updated ML services, predating the ChatGPT era but directly anticipating its methodological challenges.

2.2 Evaluation Integrity, Contamination, and Reproducibility

A closely related strand of literature interrogates whether apparent version-to-version performance changes reflect genuine capability shifts or artifacts of evaluation methodology. Aiyappa, An, Kwak, and Ahn (2023) argue that because ChatGPT is closed-source and continuously updated via RLHF, and because its training data provenance is undisclosed, standard benchmark evaluations risk data contamination — the benchmark itself, or semantically similar material, may have entered a later training update, inflating apparent performance relative to a genuinely uncontaminated baseline. A broader survey of contamination detection methods catalogs several complementary strategies for this problem, including plotting performance against the temporal release date of an evaluation dataset on the logic that datasets released after a model's training cutoff cannot have contaminated that particular snapshot (see Section 3.1.3 of the survey; the authors note that Aiyappa et al. were among the first to raise this concern systematically, and that subsequent work such as Roberts et al.'s analysis of code-generation datasets released before and after model training cutoffs extends the same logic to a different domain). Together, these works imply that any credible drift-tracking methodology must explicitly control for contamination, since a positive drift signal (apparent improvement) is confounded with the possibility that a benchmark has simply been absorbed into a later training run.

The reliability of automated evaluation itself has also come under scrutiny as LLMs increasingly serve as their own evaluators. Zheng et al. (2023) introduced the “LLM-as-a-judge” paradigm, using strong models

such as GPT-4 to score open-ended chatbot responses, and validated this approach against human preference data via the MT-Bench and Chatbot Arena benchmarks, finding over 80% agreement between LLM judges and human raters — comparable to inter-human agreement. Chatbot Arena was subsequently formalized as a standing, crowdsourced, pairwise-comparison platform for ranking LLMs by human preference (Chiang et al., 2024). While LLM-as-a-judge protocols make continuous, large-scale evaluation logistically feasible, Zheng et al. also documented systematic biases — position bias, verbosity bias, and self-enhancement bias, in which a judge model favors outputs resembling its own — that are directly relevant to drift tracking: if the judge model is itself subject to version updates, the evaluation instrument is nonstationary, compounding the difficulty of isolating drift in the model under test.

2.3 Concept Drift in Classical Machine Learning

The vocabulary of “drift” predates LLMs by decades and originates in the streaming-data and online-learning literature. Gama, Žliobaitė, Bifet, Pechenizkiy, and Bouchachia (2014) provide the canonical survey of concept drift adaptation, defining concept drift as an unforeseen change in the statistical relationship between input and target variables in a data stream, and cataloging detection strategies (e.g., statistical process control, windowing methods) and adaptation strategies (active adaptation triggered by a detector versus passive, continuous adaptation such as ensemble reweighting). Lu, Liu, Dong, Gu, Gama, and Zhang (2019) extend this survey with a taxonomy of drift types — sudden, gradual, incremental, and recurring — that maps naturally onto observed LLM update patterns: a major model release (e.g., GPT-3.5 to GPT-4) resembles sudden drift, while continued RLHF fine-tuning within a model family resembles gradual or incremental drift. However, the classical concept-drift framework assumes a fixed model whose environment shifts; LLM performance drift inverts this assumption, since the environment (the evaluation task) is often held fixed by the auditor while the model itself is silently replaced — a distinction we return to in Section 5.

2.4 Model Documentation and Transparency

A final relevant strand concerns standardized documentation for machine learning systems. Mitchell et al. (2019) proposed “model cards” as short, structured documents accompanying a released model, reporting intended use cases, evaluation results disaggregated by relevant subgroups, and known limitations, with the explicit goal of improving transparency and comparability across models and versions. Model cards, together with related documentation frameworks referenced in that work, provide a natural template for the kind of standardized, versioned reporting that the drift-tracking literature argues is currently missing for commercial LLM APIs (see Section 6). Foundation model providers’ own technical reports — for example, OpenAI’s GPT-4 technical report (OpenAI, 2023) — offer some documentation of training and evaluation at release time but, as Chen et al. (2023) and Aiyappa et al. (2023) both note, provide little to no ongoing disclosure of subsequent silent updates, leaving independent longitudinal audits as the primary source of drift evidence.

3. Main Thematic and Technical Discussion

3.1 Dimensions of Drift

Drawing on the literature reviewed above, LLM performance drift can be decomposed into at least four partially overlapping dimensions.

- **Capability drift:** changes in task accuracy on well-defined problems with verifiable answers, such as arithmetic, factual question answering, or code correctness. Chen et al.'s (2023) prime-number and code-executability results are canonical examples, and are notable for showing large-magnitude, non-monotonic swings within a matter of months.
- **Safety and refusal drift:** changes in a model's willingness to engage with sensitive, harmful, or dual-use prompts. Chen et al. (2023) found that GPT-4's direct-answer rate on sensitive questions dropped substantially between March and June 2023, which can reflect either intentional safety tuning or an unintended side effect of broader alignment updates.
- **Stylistic and formatting drift:** changes in output structure, verbosity, or formatting conventions that do not necessarily reflect a change in underlying “knowledge” but can nonetheless break downstream automated pipelines — for instance, GPT-4's tendency to wrap code in triple quotes in later 2023 snapshots, which caused previously executable code outputs to fail automated execution (Chen et al., 2023).
- **Instruction-following drift:** changes in a model's fidelity to explicit formatting or behavioral instructions in the prompt. Chen et al. (2023) present evidence that a decline in instruction-following ability is a common underlying factor linking several of the specific drifts they observe, suggesting that some apparent task-specific regressions may share a single upstream cause.

These dimensions are not independent. A single underlying change — for example, an RLHF update that increases a reward model's preference for cautious, hedged responses — can simultaneously manifest as safety drift (more refusals), stylistic drift (more hedging language), and capability drift (lower accuracy on tasks where confident, direct answers are scored as correct). This entanglement is one reason single-metric drift summaries are potentially misleading and multi-metric, scenario-based evaluation designs such as HELM (Liang et al., 2022) are valuable for drift tracking specifically, not only for one-off benchmarking.

3.2 Candidate Mechanisms Underlying Drift

The literature converges on several non-exclusive technical mechanisms that plausibly generate the drift patterns documented empirically, though — as we discuss in Section 5 — the closed nature of commercial APIs means that causal attribution to any single mechanism is rarely possible from the outside.

- **Continued instruction tuning and RLHF:** providers iteratively fine-tune deployed models on new human feedback data to improve helpfulness and reduce harmful outputs (Ouyang et al., 2022). Because RLHF optimizes a learned reward model rather than ground-truth task correctness, repeated rounds of RLHF can trade off narrow task accuracy against broader preference alignment, a dynamic consistent with the simultaneous improvements and regressions reported by Chen et al. (2023) and Tu et al. (2024).
- **System prompt and product-layer changes:** consumer products such as ChatGPT interpose a system prompt and additional product-layer logic (e.g., content filters, tool-routing) between the user and the underlying model weights. Because independent auditors typically query the product endpoint rather than raw model weights, observed “model drift” can partially or wholly reflect changes at this product layer rather than changes to the model itself — a confound explicitly acknowledged in the drift literature (Chen et al., 2023).

- Quantization and inference-serving changes: to control inference cost at scale, providers may alter numerical precision, batching strategy, or serving infrastructure for a nominally unchanged model checkpoint. Such changes can subtly alter output distributions even absent any change to the trained weights themselves, though this mechanism is comparatively under-studied relative to fine-tuning-based explanations in the published drift literature.
- Benchmark and data contamination feedback loops: because user interactions with deployed models are sometimes incorporated into subsequent training data, and because benchmark content circulates publicly, later model versions may have partially memorized specific evaluation items, producing apparent “improvement” that is properly attributed to contamination rather than generalizable capability gain (Aiyappa et al., 2023).

3.3 Drift as a Software Reliability Problem

Beyond its machine-learning dimensions, drift is usefully framed as a software dependency-management problem. Traditional software engineering practice relies on semantic versioning, changelogs, and regression test suites to give downstream developers guarantees — or at least advance warning — about behavioral changes in a dependency. Commercial LLM APIs typically offer only coarse-grained version identifiers (e.g., dated model snapshots or a floating “latest” alias), and, as Chen et al. (2023) emphasize, it is often “opaque when and how” these updates occur. This creates a mismatch between the reliability guarantees developers expect from a software dependency and the actual behavioral stability of the LLM component, which several authors argue necessitates continuous, application-level regression testing analogous to that used for conventional software (Chen et al., 2021; Chen et al., 2023).

4. Current Approaches and Developments

4.1 Longitudinal Benchmarking Infrastructures

Several projects now maintain persistent, repeatedly-sampled evaluation infrastructure specifically to support drift tracking rather than one-off benchmarking. ChatLog (Tu et al., 2024) is the most directly drift-oriented example, combining a monthly-refreshed battery of 21 established NLP benchmarks with a daily-refreshed set of long-form generation prompts, explicitly designed to expose both coarse (monthly) and fine-grained (daily) temporal patterns. Chatbot Arena (Chiang et al., 2024) offers a complementary, continuously operating crowdsourced comparison platform; because new models and model versions are added to the arena as they become available, longitudinal Elo-style rating trajectories implicitly capture relative drift across a model family over time, even though the platform was not originally designed as a drift-detection tool.

4.2 Holistic and Multi-Metric Evaluation Frameworks

HELM (Holistic Evaluation of Language Models; Liang et al., 2022) addresses a problem closely related to drift tracking: the tendency of narrow, single-metric benchmarks to obscure trade-offs across accuracy, robustness, fairness, bias, toxicity, and efficiency. By evaluating a broad set of models across a fixed taxonomy of scenarios and metrics, and by publicly releasing all raw prompts and completions, HELM provides both a template and reusable infrastructure that later drift-tracking work can, in principle, repeat at successive points in time to obtain multi-dimensional drift profiles rather than single-number

performance summaries. The HELM authors explicitly frame the project as a “living benchmark,” intended to be continuously updated — a design philosophy directly compatible with, though not identical to, longitudinal drift monitoring.

4.3 LLM-as-a-Judge and Preference-Based Monitoring

Because many dimensions of drift (tone, helpfulness, safety posture) resist simple exact-match scoring, LLM-as-a-judge protocols (Zheng et al., 2023) have become a practical component of monitoring pipelines, enabling scalable comparison of model outputs across versions on open-ended tasks. Industry monitoring tools built around this idea — sometimes marketed under the label “prompt drift” or “behavioral drift” monitoring — typically version-control prompts, log outputs continuously, and use an automated judge (frequently another LLM) to flag statistically significant deviations from a historical baseline, operationalizing the distinction between deliberate prompt regressions (caused by a developer’s own prompt edits) and externally caused drift (caused by an undisclosed provider-side model update).

4.4 Contamination-Aware Evaluation

A parallel body of tooling addresses the confound between genuine drift and contamination artifacts. Time-stratified evaluation — plotting accuracy against the public release date of the evaluation data, so that data released after a given model snapshot’s training cutoff cannot have contaminated that snapshot — is the most widely used strategy, and is discussed as a central methodological tool in recent surveys of contamination detection methods. Complementary approaches include perturbation-based membership tests (probing whether a model reproduces benchmark items verbatim or with only superficial variation) and the construction of continuously refreshed, never-before-public evaluation sets, a strategy conceptually related to ChatLog’s own daily-refreshed prompt set (Tu et al., 2024).

4.5 Documentation-Based Approaches

Finally, several proposals extend model-card-style documentation (Mitchell et al., 2019) toward versioned, update-aware reporting: rather than a single static document accompanying a model’s initial release, a versioned model card would be re-issued, or incrementally amended, at each significant update, explicitly reporting what changed and on which evaluation axes. While this practice is not yet standard among major commercial LLM providers, it represents a natural convergence point between the model-documentation literature and the drift-tracking literature reviewed in Section 2.

5. Research Challenges and Limitations

Despite growing interest, the empirical study of LLM performance drift faces several substantial methodological and infrastructural obstacles.

5.1 Provider Opacity and Confounded Update Layers

As noted throughout Sections 2–3, independent researchers typically cannot distinguish changes to model weights from changes to system prompts, safety filters, retrieval augmentation, or serving infrastructure, because commercial APIs expose only an integrated product endpoint (Chen et al., 2023). This confound limits the causal interpretability of observed drift: a measured behavioral change is, at best, attributable

to “the service as a whole,” not to any specific mechanism, which in turn limits the actionability of drift findings for both providers and downstream developers.

5.2 Benchmark Contamination and Reproducibility

Because closed models are continuously retrained on data that may include prior evaluation content or its derivatives, apparent performance changes across versions are confounded with contamination (Aiyappa et al., 2023). This problem is compounded by the reproducibility concerns inherent to evaluating a nonstationary target: a benchmark score reported for “GPT-4” in one month may not be reproducible against the model actually served under that name in a subsequent month, undermining the basic scientific expectation that published results can be independently verified (Chen et al., 2023).

5.3 Evaluator Nonstationarity

The increasing reliance on LLM-as-a-judge protocols for scalable evaluation (Zheng et al., 2023) introduces a second-order version of the same problem: if the judge model is itself periodically updated, drift in the evaluator can be conflated with drift in the model under evaluation. Known judge biases — position, verbosity, and self-enhancement bias — further complicate longitudinal comparison, since a judge model’s biases may themselves shift across its own versions.

5.4 Limited Scope of Existing Longitudinal Studies

Existing longitudinal audits, while foundational, cover a narrow slice of the LLM landscape: most published work focuses on OpenAI’s GPT-3.5 and GPT-4 accessed via ChatGPT (Chen et al., 2023; Tu et al., 2024), leaving open questions about whether similar drift magnitudes and patterns generalize to other major providers, to open-weight models subject to community fine-tuning, or to non-English and multimodal use cases. Most studies also compare only two or a small number of discrete snapshots rather than establishing a continuous, statistically characterized drift trajectory of the kind long available in classical concept-drift research (Gama et al., 2014; Lu et al., 2019).

5.5 Absence of Standardized Metrics and Reporting

Unlike classical concept drift, which benefits from decades of standardized statistical detectors and evaluation protocols (Gama et al., 2014), the LLM drift literature has not yet converged on standardized magnitude metrics, significance thresholds, or reporting formats, making cross-study comparison difficult. Model cards (Mitchell et al., 2019) offer a documentation template but were designed for single-release reporting rather than continuous, update-aware disclosure, and have not been widely adapted for this purpose by commercial LLM providers.

6. Research Gaps and Future Directions

6.1 Causal Attribution of Drift to Specific Update Mechanisms

A central gap is the absence of methods capable of attributing an observed behavioral shift to a specific cause — fine-tuning, RLHF, system-prompt changes, quantization, or serving infrastructure — rather than treating the API endpoint as an unanalyzable black box. Progress here likely requires closer collaboration

with, or voluntary disclosure from, model providers, or the development of statistical decomposition techniques capable of separating product-layer from weight-level changes using only external query access, extending the API-shift detection methodology introduced by Chen et al. (2021) toward causal rather than purely descriptive analysis.

6.2 Shared, Continuously Maintained Drift-Monitoring Infrastructure

Building on ChatLog's (Tu et al., 2024) precedent, the field would benefit from community-maintained, continuously updated “canary” benchmarks explicitly designed for drift detection — combining never-previously-public items (to control for contamination), multi-dimensional scenario coverage in the spirit of HELM (Liang et al., 2022), and standardized statistical drift-significance testing adapted from the concept-drift literature (Gama et al., 2014). Such infrastructure would ideally be provider-agnostic and openly governed, analogous to how Chatbot Arena (Chiang et al., 2024) operates as shared, cross-provider evaluation infrastructure for preference ranking.

6.3 Standardized, Versioned Model Documentation

Extending model cards (Mitchell et al., 2019) into a versioned reporting format — issued or amended at each significant model update, with explicit before/after metrics on a fixed evaluation suite — would directly address the reproducibility and transparency gaps identified in Section 5. Future work could specify a concrete schema for such “update cards,” analogous to software changelogs, and evaluate whether voluntary provider adoption is feasible or whether external, independently maintained “shadow” documentation (produced by third-party auditors) is a more realistic near-term path.

6.4 Broader Coverage Across Providers, Modalities, and Languages

Given that existing longitudinal evidence is concentrated on English-language, text-only evaluation of a small number of OpenAI model snapshots (Chen et al., 2023; Tu et al., 2024), a clear gap exists in understanding whether drift magnitude, direction, and mechanism generalize across providers (e.g., Anthropic, Google, Meta, and open-weight model families), across modalities (vision-language and audio-language models), and across languages other than English, where safety tuning and instruction-following behavior may evolve on different trajectories.

6.5 Integrating Drift Tracking with Evaluation-Integrity Research

Because contamination and drift are empirically confounded (Aiyappa et al., 2023), future methodological work should more tightly integrate time-stratified, contamination-aware evaluation design directly into drift-tracking studies from the outset, rather than treating contamination control as a separate concern, so that reported drift magnitudes can be more confidently interpreted as reflecting genuine behavioral change rather than benchmark leakage.

6.6 Downstream Reliability Engineering

Finally, relatively little published work translates drift findings into concrete reliability-engineering practices for application developers — for example, automated regression test suites that flag when a production LLM dependency's outputs on a fixed evaluation set change beyond a specified tolerance, in

direct analogy to conventional software regression testing. Developing, validating, and standardizing such tooling represents a practically important and currently underexplored direction connecting the machine-learning drift literature to software reliability engineering.

7. Conclusion

The evidence reviewed in this paper indicates that performance drift is a real, measurable, and practically consequential feature of commercially deployed large language models, documented most rigorously in longitudinal audits of GPT-3.5 and GPT-4 (Chen et al., 2023; Tu et al., 2024) and situated within a broader literature on evaluation integrity (Aiyappa et al., 2023; Zheng et al., 2023), classical concept drift (Gama et al., 2014; Lu et al., 2019), and model documentation (Mitchell et al., 2019). Drift manifests across capability, safety, stylistic, and instruction-following dimensions, is not uniformly degenerative, and is driven by a combination of continued fine-tuning, RLHF updates, product-layer changes, and serving-infrastructure decisions that are rarely disentangled by external observers. Current monitoring approaches — longitudinal benchmark suites, holistic multi-metric frameworks, LLM-as-a-judge protocols, and contamination-aware evaluation — provide a useful but incomplete foundation, constrained by provider opacity, evaluator nonstationarity, and the absence of standardized, versioned reporting analogous to software changelogs. Closing these gaps — through causal attribution methods, shared drift-monitoring infrastructure, versioned model documentation, broader cross-provider and cross-lingual coverage, and tighter integration with contamination-aware evaluation — is necessary if organizations are to build reliable, auditable systems atop foundation models that, by design, continue to change beneath them.

References

- Aiyappa, R., An, J., Kwak, H., & Ahn, Y.-Y. (2023). Can we trust the evaluation on ChatGPT? In Proceedings of the 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP 2023) (pp. 47–54). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.trustnlp-1.5> (arXiv:2303.12767)
- Chen, L., Cai, T., Zaharia, M., & Zou, J. (2021). Did the model change? Efficiently assessing machine learning API shifts. arXiv. arXiv:2107.14203
- Chen, L., Zaharia, M., & Zou, J. (2023). How is ChatGPT's behavior changing over time? arXiv. arXiv:2307.09009
- Chen, L., Zaharia, M., & Zou, J. (2024). How is ChatGPT's behavior changing over time? Harvard Data Science Review, 6(2). <https://doi.org/10.1162/99608f92.5317da47>
- Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., Zhu, B., Zhang, H., Jordan, M., Gonzalez, J. E., & Stoica, I. (2024). Chatbot Arena: An open platform for evaluating LLMs by human preference. In Proceedings of the 41st International Conference on Machine Learning (ICML 2024). arXiv:2403.04132
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), Article 44, 1–37. <https://doi.org/10.1145/2523813>
- Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C. D., Ré, C., Acosta-Navas, D.,

- Hudson, D. A., ... Koreeda, Y. (2022). Holistic evaluation of language models. arXiv. <https://doi.org/10.48550/arXiv.2211.09110> (arXiv:2211.09110)
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363. <https://doi.org/10.1109/TKDE.2018.2876857>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)* (pp. 220–229). Association for Computing Machinery. <https://doi.org/10.1145/3287560.3287596> (arXiv:1810.03993)
- OpenAI. (2023). GPT-4 technical report. arXiv. arXiv:2303.08774
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Aspell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. arXiv:2203.02155
- Tu, S., Li, C., Yu, J., Wang, X., Hou, L., & Li, J. (2024). ChatLog: Carefully evaluating the evolution of ChatGPT across time. arXiv. arXiv:2304.14106
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. arXiv:2306.05685